

Sustento teórico del CRB polar NLOS

Matemática de nuestra formulación e implementación

Obstacle Detection Study

August 19, 2026

Abstract

Este documento es el **sustento teórico** del análisis CRB que realizamos en el repositorio (`simulation_polar_clean.ipynb`, `crb_polar_functions.py`). La estructura es *similar* a la del writeup de Active Corner Camera (parametrización polar, Poisson, Fisher, cotas 3σ), pero **no es una copia**: adoptamos un modelo de medida discreto (suma sobre triángulos y bins), estimamos $\psi = [\rho, \varphi, h]^T$ (Fisher 3×3), tratamos el ancho w como fijo, y obtenemos derivadas por diferencias finitas. El objetivo es poder citar este texto como respaldo matemático de lo que el código calcula.

Contents

1	Objetivo del análisis	2
2	Parametrización de la escena	2
2.1	Pose polar	2
2.2	Ancho fijo; altura estimada	3
3	Modelo de medida discreto	3
3.1	Cubo transitorio y tasa de Poisson	3
3.2	Construcción de y : suma como integral	3
3.3	Oclusión / “sombra” (cómo se calcula)	4
4	Verosimilitud, score y Fisher	4
4.1	Log-verosimilitud	4
4.2	Score y matriz de información	4
5	Cota de Cramér–Rao y regiones de incertidumbre	4
5.1	CRB	4
5.2	Elipse 3σ en (ρ, φ)	5
5.3	Pushforward opcional a metros en el suelo	5
6	Derivadas: aproximación por diferencias finitas	5
7	Fondo y oclusión en la CRB	5
8	Cadena matemática implementada	6
9	Qué es igual y qué no respecto al writeup	6
10	Figuras de regiones CRB (con h y sin h)	6

11 Qué se usa para generar los plots CRB	6
11.1 Piezas que intervienen	7
11.2 Paso 1 — tasa esperada g	7
11.3 Paso 2 — Jacobiano por diferencias finitas	8
11.4 Paso 3 — Fisher y CRB	8
11.5 Paso 4 — elipse 3σ y plot	9
11.6 Dos plots (llamada al grid)	9
11.7 Orquestación	9

1 Objetivo del análisis

Queremos acotar cuán precisamente se puede localizar una faceta oculta en el plano del suelo a partir de un histograma SPAD×tiempo de fotones, bajo el modelo de Poisson. La herramienta es la **cota de Cramér–Rao** (CRB) asociada a la matriz de información de Fisher de la medida esperada.

En nuestra formulación:

1. la escena se parametriza en polares (ρ, φ) ;
2. la medida esperada g se obtiene de un simulador de transporte discreto (rebote faceta $\rightarrow$ FOV);
3. el Jacobiano $\partial g / \partial \psi$ se aproxima por diferencias finitas sobre **todos** los bins;
4. reportamos σ_ρ , σ_φ , σ_h y regiones 3σ marginales en (ρ, φ) .

Observación 1 (Relación con el writeup). El writeup de Seidel et al. usa el mismo espíritu (Fisher de Poisson + oclusión por arista) con $\Psi = [\alpha, \rho, \varphi, h]$ y derivadas analíticas bajo el integrando. Nosotros **heredamos la idea**, pero el objeto matemático que analizamos es el de nuestro forward discreto y $\psi = [\rho, \varphi, h]$ (sin albedo α).

Observación 2 (Estado actual del código). La prueba canónica que reportamos **incluye la altura** h como parámetro desconocido (`estimate_height=True`): Fisher 3×3 , pasos FD $[\delta\rho, \delta\varphi, \delta h]$, y elipses 3σ **marginales** en (ρ, φ) (la incertidumbre de h ya está absorbida al invertir **I**). El ancho w permanece fijo.

2 Parametrización de la escena

2.1 Pose polar

Una faceta vertical se coloca en el plano $z = 0$ (base) con

$$x = \rho \cos \varphi, \quad y = \rho \sin \varphi, \quad \rho > 0, \quad \varphi \in (0, \pi) \quad (1)$$

en la convención del simulador (X, Y = suelo, Z = altura). El vector de parámetros **estimados** en el CRB es, por defecto,

$$\psi = \begin{bmatrix} \rho \\ \varphi \\ h \end{bmatrix}, \quad (2)$$

donde h es la altura vertical de la faceta. La Fisher resultante es 3×3 y reportamos σ_ρ , σ_φ y σ_h . Las elipses en el plano (ρ, φ) usan el bloque 2×2 **marginal** de **I**⁺ (altura desconocida).

La normal media de la faceta se orienta hacia la arista, $\hat{n} \approx [-\cos \varphi, -\sin \varphi, 0]$, para que mire al lado visible/láser.

2.2 Ancho fijo; altura estimada

El ancho horizontal w permanece como parámetro de escena fijo (no se diferencia). La altura h sí entra en $\boldsymbol{\psi}$ y se acota con $\sigma_h = \sqrt{[\mathbf{I}^+]_{22}}$. En el mesh (antes del pitch $\approx \pi/2$)

$$\text{scale}_x = \frac{w}{\text{extent}_X}, \quad \text{scale}_y = \frac{h}{\text{extent}_Y}, \quad (3)$$

de modo que h es independiente de w . Por defecto $h_0 = 1$ m.

Opcionalmente se puede fijar h (`estimate_height=False`) y recuperar el CRB 2×2 solo en (ρ, φ) .

3 Modelo de medida discreto

3.1 Cubo transitorio y tasa de Poisson

Sea $y(\boldsymbol{\psi}) \in \mathbb{R}^{H \times W \times T}$ el histograma esperado de fotones (sin ruido de muestreo) producido por el forward. Con fondo (dark counts / ambient) $b \geq 0$,

$$g(\boldsymbol{\psi}) = y(\boldsymbol{\psi}) + b, \quad \mathbf{g} = \text{vec}(g(\boldsymbol{\psi})) \in \mathbb{R}^N, \quad N = HWT. \quad (4)$$

Cada bin $q = 1, \dots, N$ se modela como

$$X_q \sim \text{Poisson}(g_q(\boldsymbol{\psi})) \quad \text{independientes.} \quad (5)$$

3.2 Construcción de y : suma como integral

El writeup escribe y_q como integral de una radiancia L_h sobre el píxel y el bin temporal. Nosotros **definimos** y mediante una cuadratura asociada al mesh:

1. Descomponer la faceta (y opcionalmente paredes) en triángulos de centro $\mathbf{c}$, área A y normal $\hat{\mathbf{n}}$.
2. Para cada muestra espacial $\mathbf{p}_f$ del FOV (centros de SPAD y subpíxeles), acumular

$$\Delta y \propto I_{\text{laser}} A [\hat{\mathbf{n}} \cdot \hat{\mathbf{u}}_1]_+ \cdots [\hat{\mathbf{n}}_f \cdot \hat{\mathbf{u}}_4]_+ \frac{1}{4\pi^2 d_1^2 d_2^2}, \quad (6)$$

con $d_1 = \|\mathbf{p}_t - \mathbf{c}\|$, $d_2 = \|\mathbf{p}_f - \mathbf{c}\|$.

3. Asignar el retardo a un bin

$$j = \left\lceil \frac{d_1 + d_2}{c \Delta t} \right\rceil \quad (7)$$

$(\Delta t = \text{bin_size})$.

En notación compacta,

$$y_q = \sum_{\Delta} \sum_{\text{sub} \in P_i} \mathbf{1}\{\text{visible}\} \mathbf{1}\{j = q_t\} \Delta y_{\Delta, \text{sub}}. \quad (8)$$

Esto es nuestra “integral” de medida: una suma de Riemann sobre elementos de superficie y muestras espaciales, con pulso idealizado como masa en un solo bin.

3.3 Oclusión / “sombra” (cómo se calcula)

La esquina oculta caminos faceta→FOV que no cruzan la arista visible. En el código (`simulation`), para cada triángulo de centro $\mathbf{c} = (c_x, c_y, c_z)$ y cada muestra de cámara $\mathbf{p}_f = (p_x, p_y, p_z)$:

1. Se traza la recta en el plano XY entre $\mathbf{c}$ y $\mathbf{p}_f$:

$$m = \frac{c_y - p_y}{c_x - p_x}, \quad b = c_y - m c_x.$$

2. Intersección con la arista $x = 0$:

$$\mathbf{xint} = -b/m$$

(NaN si el denominador o m son nulos).

3. Máscara de caminos *no ocluidos*:

$$\mathbf{noc} = (\mathbf{xint} \text{ finito}) \wedge (\mathbf{xint} > 0). \quad (9)$$

4. Solo los índices con $\mathbf{noc}=\text{True}$ reciben intensidad Δy en el histograma. El resto queda en $y = 0$ (píxel / bin en sombra para esa geometría).

En la CRB **no descartamos** esos bins: $g = \text{vec}(y + b)$ incluye $y \approx 0$ en sombra. Al perturbar φ o h por FD, un bin puede pasar de sombra a iluminado (o al revés) y eso aparece en $\mathbf{J}$. Eso es “incluir los píxeles sombreados en las derivadas”.

4 Verosimilitud, score y Fisher

4.1 Log-verosimilitud

Con observaciones x_q y tasa $g_q(\boldsymbol{\psi})$,

$$\ell(\boldsymbol{\psi}) = \sum_{q=1}^N \left(x_q \log g_q(\boldsymbol{\psi}) - g_q(\boldsymbol{\psi}) - \log(x_q!) \right). \quad (10)$$

4.2 Score y matriz de información

Derivando respecto a $\boldsymbol{\psi}_m$ y tomando esperanza bajo el modelo (5),

$$I_{mk}(\boldsymbol{\psi}) = \sum_{q=1}^N \frac{1}{g_q(\boldsymbol{\psi})} \frac{\partial g_q}{\partial \boldsymbol{\psi}_m} \frac{\partial g_q}{\partial \boldsymbol{\psi}_k}, \quad m, k \in \{\rho, \varphi, h\}. \quad (11)$$

En forma matricial, con Jacobiano $\mathbf{J} \in \mathbb{R}^{N \times 3}$ cuyas columnas son $\partial \mathbf{g} / \partial \rho$, $\partial \mathbf{g} / \partial \varphi$ y $\partial \mathbf{g} / \partial h$,

$$\mathbf{I} = \mathbf{J}^\top \text{diag}(1/g) \mathbf{J}. \quad (12)$$

Esta es la misma estructura Fisher–Poisson del writeup; cambia el dominio de parámetros (3 en lugar de 4, sin albedo) y cómo se obtienen g y $\mathbf{J}$.

Observación 3 (Regularización numérica). Si $g_q = 0$, el término $1/g_q$ no está definido. En la implementación usamos $g_q \leftarrow \max(g_q, \varepsilon)$ con $\varepsilon \ll 1$ (`fisher_eps`).

5 Cota de Cramér–Rao y regiones de incertidumbre

5.1 CRB

Para un estimador insesgado $\hat{\boldsymbol{\psi}}$ de $\boldsymbol{\psi}$,

$$\text{Cov}(\hat{\boldsymbol{\psi}}) \succeq \mathbf{I}(\boldsymbol{\psi})^{-1}, \quad (13)$$

en el sentido de matrices semidefinidas positivas (usamos la pseudoinversa $\mathbf{I}^+$ si $\mathbf{I}$ es mal condicionada). Definimos

$$\sigma_\rho = \sqrt{[\mathbf{I}^+]_{00}}, \quad \sigma_\varphi = \sqrt{[\mathbf{I}^+]_{11}}, \quad \sigma_h = \sqrt{[\mathbf{I}^+]_{22}}, \quad \sigma_{\text{tang}} = \rho \sigma_\varphi. \quad (14)$$

σ_{tang} convierte la incertidumbre angular en metros de arco en la pose nominal.

5.2 Elipse 3σ en (ρ, φ)

Sea Σ el bloque 2×2 **marginal** de $\mathbf{I}^+$ en (ρ, φ) (altura desconocida ya absorbida al invertir la Fisher 3×3). Con autodescomposición $\Sigma = U \text{diag}(\lambda) U^\top$ y $k = 3$,

$$\begin{bmatrix} \rho(t) \\ \varphi(t) \end{bmatrix} = \begin{bmatrix} \rho_0 \\ \varphi_0 \end{bmatrix} + U \text{diag}(k\sqrt{\lambda}) \begin{bmatrix} \cos t \\ \sin t \end{bmatrix}, \quad t \in [0, 2\pi). \quad (15)$$

Esta curva es una elipse en el plano de parámetros (ρ, φ) . Al dibujarla en ejes polares (radio = ρ , ángulo = φ) se obtienen las “burbujas” análogas a las del writeup — **sin** proyectar antes a (x, y) .

Esa es la visualización canónica de nuestro estudio (use_physical_region=False).

5.3 Pushforward opcional a metros en el suelo

Si se desea interpretar la incertidumbre en coordenadas cartesianas,

$$\mathbf{A} = \begin{bmatrix} \cos \varphi_0 & -\rho_0 \sin \varphi_0 \\ \sin \varphi_0 & \rho_0 \cos \varphi_0 \end{bmatrix}, \quad \Sigma_{xy} = \mathbf{A} \Sigma \mathbf{A}^\top. \quad (16)$$

La elipse 3σ en (x, y) es la región espacial linealizada. Reparametrizar esa curva a polares para un plot polar la **deforma** (apariciencia de “gota”) cuando σ es grande: no es un error de Fisher, es un efecto de visualización.

6 Derivadas: aproximación por diferencias finitas

El forward $g(\psi)$ incluye máximos, techos de bin y la máscara **noc**: no es diferenciable en todos los puntos. Por ello no usamos derivadas analíticas bajo un integrando continuo, sino el Jacobiano numérico

$$\frac{\partial g}{\partial \psi_m} \approx \frac{g(\psi_0 + \delta_m \mathbf{e}_m) - g(\psi_0 - \delta_m \mathbf{e}_m)}{2\delta_m}, \quad (17)$$

$m \in \{\rho, \varphi, h\}$ (diferencias centrales, $\mathcal{O}(\delta^2)$).

Opcionalmente, extrapolación de Richardson de orden $\mathcal{O}(\delta^4)$ sobre el mismo forward.

Observación 4. Al perturbar φ o h , píxeles pueden pasar de iluminados a ocluidos (o al revés). El FD captura ese salto de frontera en $\mathbf{J}$: es el análogo numérico de diferenciar a través del Heaviside del writeup.

Costo típico (central, 3 parámetros): 7 evaluaciones de g por pose (nominal + 2 por parámetro).

7 Fondo y oclusión en la CRB

Como I_{mk} pesa cada bin con $1/g_q$ y $g_q = y_q + b$, aumentar b diluye la información y empeora la CRB.

La oclusión se calcula en el forward (Sec. 3.3). En la CRB usamos **todos** los bins: sombras entran como $y \approx 0$ y, al diferenciar, como saltos de frontera en $\mathbf{J}$. No hay paso que descarte píxeles sombreados antes del Fisher.

8 Cadena matemática implementada

$$\begin{aligned}
\psi_0 &\xrightarrow{\text{forward discreto}} y_0, \quad \mathbf{g}_0 = \text{vec}(y_0 + b) \\
&\xrightarrow{\text{FD central/Richardson}} \mathbf{J} \approx \left. \frac{\partial \mathbf{g}}{\partial \psi^\top} \right|_{\psi_0} \\
&\xrightarrow{\text{Fisher Poisson}} \mathbf{I} = \mathbf{J}^\top \text{diag}(1/\mathbf{g}_0) \mathbf{J} \\
&\xrightarrow{\text{CRB}} \Sigma = \mathbf{I}^+, \quad \text{elipse } 3\sigma \text{ en } (\rho, \varphi).
\end{aligned} \tag{18}$$

Esto es exactamente lo que orquesta `compute_crb_polar` (y `compute_crb_grid_polar` sobre una malla de poses).

9 Qué es igual y qué no respecto al writeup

	Writeup (PDF)	Nuestra formulación
Parámetros	$[\alpha, \rho, \varphi, h]$	$[\rho, \varphi, h]$; w fijo
Medida y_q	$\iint L_h d\mathbf{p} dt$	Suma (8) sobre mesh + bins
Oclusión	Heaviside analítico	Máscara <code>noc/xint</code>
$\partial g / \partial \psi$	Derivada bajo el integrando	Diferencias finitas
Fisher	$\sum (\partial_m g)(\partial_k g) / g$	Idém, eq. (11)–(12)
Regiones	Burbujas 3σ en polar	Elipse (15) en (ρ, φ)

10 Figuras de regiones CRB (con h y sin h)

Misma malla de poses

$$\rho \in \{0,5, 1,0, 1,5\} \text{ m}, \quad \varphi \in \{30^\circ, 60^\circ, 90^\circ, 120^\circ, 150^\circ\},$$

$w = 0,5 \text{ m}$, geometría $h = 1,0 \text{ m}$, fondo $b = 0,1$, FD central, elipses 3σ en (ρ, φ) (`use_physical_region=False`).

Dos variantes (el título del PNG indica qué se estima):

- **CRB**(ρ, φ, h) (`plots/crb_standard_regions.png`): se estiman ρ , φ y h ; Fisher 3×3 ; elipse marginal en (ρ, φ) .
- **CRB**(ρ, φ) (`plots/crb_standard_regions_fixed_h.png`): solo ρ y φ ; h fija en la geometría; Fisher 2×2 .

En general las elipses de **CRB**(ρ, φ, h) son más grandes que las de **CRB**(ρ, φ) (peor cota: un parámetro extra desconocido).

Regenerar ambas: `python regenerate_crb_standard_regions.py`.

11 Qué se usa para generar los plots CRB

Esta sección describe **solo la cadena CRB** (derivadas, Fisher, cota, elipses). El simulador se trata como una caja negra `simulation`(ρ, φ, w, h) $\mapsto y$.

CRB(ρ, φ, h) — regiones 3σ en (ρ, φ)

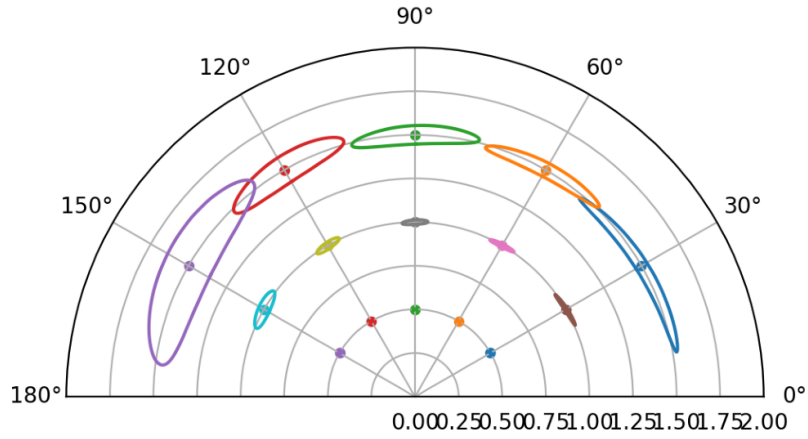


Figure 1: CRB(ρ, φ, h): regiones 3σ en (ρ, φ) . Archivo: `plots/crb_standard_regions.png`.

11.1 Piezas que intervienen

Pieza	Para qué sirve en el CRB
<code>simulation(...)</code> (notebook)	Da el histograma esperado y en una pose. No calculamos derivadas aquí.
<code>forward_g_polar</code>	Forma $g = y + b$ (vector de todos los bins SPAD $\times$ tiempo).
<code>finite_difference_jacobian_polar</code>	Aproxima $\partial g / \partial \psi_m$ por diferencias finitas centrales.
<code>fisher_poisson</code>	Arma $\mathbf{I} = \mathbf{J}^T \text{diag}(1/g) \mathbf{J}$.
<code>compute_crb_polar</code>	Invierte $\mathbf{I}$, saca σ y la elipse 3σ .
<code>compute_crb_grid_polar</code>	Repite lo anterior en la malla de poses.
<code>plot_crb_regions_polar</code>	Dibuja las elipses en ejes polares y guarda el PNG.
<code>regenerate_crb_standard_regions</code>	Regenera el grid sin Jupyter (modos CRB(ρ, φ, h) y CRB(ρ, φ)).

11.2 Paso 1 — tasa esperada g

En la pose nominal ψ_0 (p. ej. $[\rho_0, \varphi_0, h_0]$):

$$y_0 = \text{simulation}(\psi_0), \quad g_0 = \text{vec}(y_0 + b).$$

b es el fondo (`background_rate`). g_0 incluye bins en sombra ($y \approx 0$). Código en `crb_polar_functions.py`:

```
# g = y + b (forward_g_polar)
y_signal, params = simulate_facet_signal_expected(...) # llama simulation
background = float(background_rate) * np.ones_like(y_signal)
g = y_signal + background
return g, y_signal, params

# desde psi = [rho, phi] o [rho, phi, h] (forward_g_from_psi)
g, y_signal, params = forward_g_polar(..., height=h, ...)
return np.asarray(g, dtype=float).ravel(), y_signal, params
```

CRB(ρ, φ) — regiones 3σ

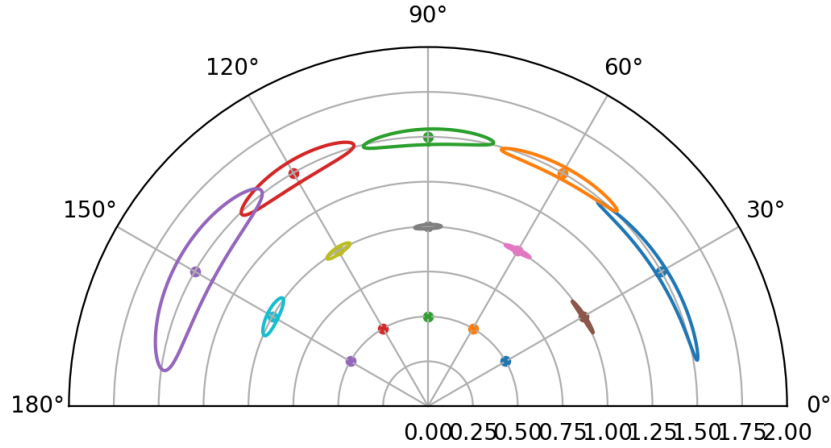


Figure 2: CRB(ρ, φ): regiones 3σ (h no se estima).
plots/crb_standard_regions_fixed_h.png.

Archivo:

11.3 Paso 2 — Jacobiano por diferencias finitas

Para cada parámetro $\psi_m \in \{\rho, \varphi\}$ o $\{\rho, \varphi, h\}$:

$$J_{.m} \approx \frac{g(\psi_0 + \delta_m \mathbf{e}_m) - g(\psi_0 - \delta_m \mathbf{e}_m)}{2\delta_m}. \quad (19)$$

Pasos típicos: $\delta\rho = 0,01$ m, $\delta\varphi = 0,5^\circ$, $\delta h = 0,01$ m.

```
# finite_difference_jacobian_polar (nucleo)
g0, y0, params0 = forward_g_from_psi(psi0, ...)
J = np.zeros((g0.size, n), dtype=float)
for m in range(n):
    dpsi = np.zeros(n); dpsi[m] = steps[m]
    g_p, _, _ = forward_g_from_psi(psi0 + dpsi, ...)
    g_m, _, _ = forward_g_from_psi(psi0 - dpsi, ...)
    J[:, m] = (g_p - g_m) / (2.0 * steps[m])
return g0, J, y0, params0
```

11.4 Paso 3 — Fisher y CRB

$$\mathbf{I} = \mathbf{J}^T \text{diag}(1/g_0) \mathbf{J}, \quad \text{CRB} = \mathbf{I}^+. \quad (20)$$

```
# fisher_poisson
g_safe = np.maximum(g, eps)
I = J.T @ ((1.0 / g_safe)[:, None] * J)

# dentro de compute_crb_polar
g0, J, y0, params0, method = compute_jacobian_polar(...)
I = fisher_poisson(g0, J, eps=fisher_eps)
```



```
CRB = np.linalg.pinv(I)
sigma_rho = np.sqrt(CRB[0, 0])
sigma_phi = np.sqrt(CRB[1, 1])
# si 3x3: sigma_height = np.sqrt(CRB[2, 2])
```

11.5 Paso 4 — ellipse 3σ y plot

Bloque 2×2 de (ρ, φ) (marginal si Fisher es 3×3):

```
# crb_region_in_polar_parameters
Sigma = CRB[np.ix_([0, 1], [0, 1])]
evals, evects = np.linalg.eigh(Sigma)
t = np.linspace(0, 2*np.pi, n_points)
circle = np.vstack([np.cos(t), np.sin(t)])
axes = k * np.sqrt(np.maximum(evals, 0))
center = np.array([rho0, phi0])
ellipse = center[:, None] + evects @ np.diag(axes) @ circle
rho_curve, phi_curve = ellipse[0], ellipse[1]

# plot_crb_regions_polar
fig, ax = plt.subplots(subplot_kw={"projection": "polar"})
for result in results:
    ax.plot(result["phi_curve_param"], result["rho_curve_param"])
    ax.scatter([result["phi0"]], [result["rho0"]], s=15)
ax.set_thetamin(0); ax.set_thetamax(180)
fig.savefig(output_path, dpi=dpi)
```

11.6 Dos plots (llamada al grid)

```
# CRB(rho, phi, h) -> plots/crb_standard_regions.png
results_h = compute_crb_grid_polar(
    ..., estimate_height=True,
    finite_difference_steps=[0.01, np.deg2rad(0.5), 0.01],
)
plot_crb_regions_polar(
    results_h, use_physical_region=False, rlim=2.0,
    title="CRB(rho, phi, h) - regiones 3 sigma",
    output_path="plots/crb_standard_regions.png",
)

# CRB(rho, phi) -> plots/crb_standard_regions_fixed_h.png
results_fixed = compute_crb_grid_polar(
    ..., estimate_height=False,
    finite_difference_steps=[0.01, np.deg2rad(0.5)],
)
plot_crb_regions_polar(
    results_fixed, use_physical_region=False, rlim=2.0,
    title="CRB(rho, phi) - regiones 3 sigma",
    output_path="plots/crb_standard_regions_fixed_h.png",
)
```

11.7 Orquestación

`compute_crb_polar` = pasos 1–4 en una pose. `compute_crb_grid_polar` = bucle en $\rho \in \{0, 5, 10, 15\}$, $\varphi \in \{30^\circ, \dots, 150^\circ\}$. El notebook o `regenerate_crb_standard_regions.py` llaman al grid y guardan el PNG.